

Euron 10주차 발표 요약 — Tokenization & 임베딩

4팀 이가은 · 김예나 · 김규리

1. 자연어 처리(NLP)와 토큰화 개요

컴퓨터는 텍스트를 직접 이해하지 못하므로 숫자 벡터로 변환해야 한다. 그 첫 단계가 토큰화(Tokenization)이다. 인간 언어는 모호성, 가변성, 구조성이라는 세 가지 특성을 지니며, 이로 인해 자동 처리가 어렵다.

처리 흐름: 사람 → 텍스트 입력 → 컴퓨터 → 토큰화 → 수치화 → 모델 학습

2. 토큰화 핵심 개념

- Corpus(말뭉치): 분석 대상이 되는 텍스트 데이터셋 (예: 뉴스 기사 모음, 리뷰 데이터셋)
- Token(토큰): 토큰화된 결과물인 개별 단위 (예: 단어, 글자, 형태소)
- Tokenization(토큰화): 텍스트를 토큰으로 분리하는 과정 — NLP의 첫 번째 단계
- Tokenizer(토큰라이저): 토큰화를 수행하는 규칙/모델. 방식에 따라 결과가 크게 달라진다.

3. 토큰라이저 구축 방법

- 공백 분할: 띄어쓰기 기준으로 분리. 가장 단순하지만 한국어·중국어에는 부적합.
- 정규 표현식: 패턴 규칙으로 토큰 분리. 특수문자·URL 등 구조적 텍스트에 유용.
- 어휘 사전 기반: 사전에 등록된 단어만 토큰으로 인식. 미등록 단어(OOV) 문제 발생 가능.
- 머신러닝 기반: 데이터에서 토큰화 규칙을 자동 학습 (예: BPE, WordPiece).

OOV(Out-of-Vocabulary) 문제

학습 어휘 사전에 없던 단어가 입력으로 들어오는 문제로, 어휘가 커질수록 Sparse data와 차원의 저주가 심화된다. 해결책으로 단어를 아는 조각으로 분해하는 하위 단어 토큰화가 도입되었다.

4. 토큰화 단위별 방법

단어 토큰화

공백(띄어쓰기) 기준으로 분리. 구두점이 붙은 'cg.'와 'cg'가 다른 토큰으로 처리되는 등 OOV 가능성이 높아진다. 품사 태깅·개체명 인식·기계번역 등에 활용된다.

글자 토큰화

띄어쓰기 + 글자 단위로 분리. 어휘 크기가 작고 OOV가 줄지만 의미 정보가 부족하다. 한국어의 경우 자모 토큰화(초성/중성/종성 분리)도 사용된다.

형태소 토큰화

의미를 가진 최소 단위인 형태소로 분리. 한국어처럼 교착어에 특히 효과적이며 품사 태깅(POS)을 통해 문맥 이해가 가능하다. 대표 라이브러리: KoNLPy(Okt — 19개 품사 / 꼬꼬마 — 56개 품사), NLTK(영어).

5. 하위 단어 토큰화

기존 형태소 분석기는 신조어·오타자·미등록 단어 처리에 취약하다. 하위 단어 토큰화는 하나의 단어를 자주 등장하는 하위 단어의 조합으로 나누어 OOV 문제를 완화하고 어휘 사전 크기를 줄인다.

BPE (Byte Pair Encoding)

가장 빈도 높은 글자 쌍을 반복적으로 병합한다. 자주 등장하는 단어는 하나의 토큰으로, 드문 단어는 여러 토큰의 조합으로 표현된다. 구글의 SentencePiece 라이브러리(SentencePieceTrainer · SentencePieceProcessor)로 구현 가능하다.

WordPiece

BPE와 유사하지만 빈도 대신 확률 기반 점수($\text{score} = f(x,y) / f(x) \cdot f(y)$)로 글자 쌍을 선택한다. Hugging Face의 tokenizers 라이브러리(WordPiece API)로 쉽게 구현할 수 있으며, 정규화(NFD 유니코드 + 소문자 변환)와 사전 토큰화를 지원한다.

6. 임베딩

워드 임베딩

단어를 벡터로 바꾸어 표현하는 방법이다. 원-핫 인코딩과 빈도 벡터화는 벡터가 희소(sparse)하다는 한계가 있다. Word2Vec, fastText 같은 분산 표현은 비슷한 의미의 단어를 가까운 벡터로 매핑하지만, 단어를 항상 같은 값으로 표현해 문맥에 따른 의미 구분이 어렵다 → 동적 임베딩으로 해결.

언어 모델

- 언어 모델: 앞에 나온 단어들을 보고 다음 단어를 예측하는 모델.
- 자기회귀 언어 모델: 이전 단어들의 정보를 이용해 다음 단어의 확률을 계산하며 순서대로 예측.
- 통계적 언어 모델: 단어의 순서와 등장 빈도를 바탕으로 예측. 마르코프 체인으로 구현되나 긴 문맥 처리에 한계.

최근 자연어 처리는 대량의 텍스트를 사전 학습한 모델(GPT, BERT 등)을 활용한다.

N-gram

연속된 N개의 단어를 묶어 다음 단어를 예측하는 기본 언어 모델. N=1(유니그램), N=2(바이그램), N=3(트라이그램). 이전 N-1개의 단어만 참고하므로 적은 데이터에서도 패턴 분석이 가능하고 관용 표현 분석에 유용하다.

TF-IDF

문서에서 중요한 단어를 찾는 방법. BoW(단순 빈도)에 단어 중요도 개념을 추가했다.

- TF(단어 빈도): 특정 단어가 문서에 등장하는 횟수.
- DF(문서 빈도): 해당 단어가 등장하는 문서 수. 많은 문서에 등장할수록 흔한 단어.
- IDF(역문서 빈도): 여러 문서에 흔하지 않은 단어일수록 높은 값 부여. $IDF = \log(\text{count}(D) / (1 + DF(t,D)))$.
- TF-IDF = TF × IDF: 특정 문서에서만 자주 등장하는 단어의 중요도를 높게 평가.
- 사이킷런의 TfidfVectorizer로 구현 가능(stop_words, ngram_range, max_df, min_df 등 파라미터 제공).